

1 Any shape you want

0:00 · 18s

- 1 **Play-doh.**
- 2 It's a fun toy: you can create any **shape** you want with it.
- 3 To me personally though, that much freedom always felt more like a **burden**.
- 4 You don't get much **assistance** from the material.
- 5 • I gravitated more towards **lego**.

2 What carries over to your next app?

0:18 · 78s

- 1 **Restrictions** and well-defined interfaces helped me be more creative.
- 2 Two **lumps** of play-doh assemble however you press them together.
- 3 **Assembly** of two bricks on the other hand is mediated by studs.
- 4 On the **surface**, lego costs you freedom.
- 5 There are **shapes** you simply cannot build.
- 6 What you get back is **reuse**, and with shaping constraints, you can spend your attention on the model instead of the material.
- 7 Much of this **applies** to what we do.
- 8 I work at a small **consultancy**, and we have built a lot of Shiny apps over the years.
- 9 Every project felt like it needed to be **taylor-made**, so we ended up solving similar problems over and over again.
- 10 **Code** bases grew large and hard to change.
- 11 **Clay**, basically.
- 12 Now how can we **improve**?
- 13 How can we have **components** carry over from one app to the next?
- 14 One way is by imposing **restrictions**.
- 15 • If we **limit** the shape by defining an interface, this enables reuse.
- 16 It lets you **focus** on what matters.
- 17 • And it applies to humans and current-gen coding **assistants** alike.

3 A block

1:36 · 29s

- 1 The core abstraction we built for blockr is a block: a well-defined **unit** of computation.
- 2 Blocks are connected by **links** and can be assembled into stacks.
- 3 • Each block has data **inputs**, user parameters and a single output.
- 4 • That is the whole **contract**.
- 5 • The block is the **brick**, the link is the stud and the stack is the model.
- 6 • The stud is our **interface**: simple and well-defined.

4 A real app

2:05 · 65s

- 1 We are using blockr at **Bristol-Myers Squibb** to explore clinical data from studies.
- 2 Our **frontend** here is built on a docking layout manager and we have every block render a panel you can move and resize freely.
- 3 These apps pull in up to one **hundred** blocks, spread over a dozen views.
- 4 And we're **rolling** out many such apps, specific to studies and their analysis needs.
- 5 **Blocks** are not the only extension point.

- 6 There are **extensions** as well:
- 7 (1) On the left, we have a **graphical** overview, showing blocks, their connections and stack membership.
- 8 (2) And on the right, an **AI** assistant, which can manipulate and interrogate our app.
- 9 Most blocks here are **general** purpose: reading data, transforming it, displaying summary tables and visualizations that support drill-down.
- 10 • (3) But one block was built specifically for this use case: the **patient profile** you can see in the middle.

5 Specialty pieces

3:10 • 26s

- 1 Our current **library** of general-purpose blocks cannot cover every use case.
- 2 Similar to **specialty** pieces that come with certain lego sets, sometimes you want a specific shape, which still comes with the standard studs.
- 3 We made it **easy** for you to do just that by integrating your own blocks into a blockr app.
- 4 • Here's what one **looks** like.

6 Anatomy of a block

3:36 • 31s

- 1 The **head** block:
- 2 (1) First are user **parameters**: here we need only one, n.
- 3 (2) It is **seeded** by the corresponding constructor argument.
- 4 (3) Next are data **inputs**: we again have only a single one: x.
- 5 (4) The data **output** is represented by an expression, so the result can be computed on demand from input data.
- 6 • (5) And finally we return **state**: this allows us to save and restore.

7 Build a block.

4:07 • 36s

- 1 Now it's your **turn**: build a block.
- 2 Either by hand, or by pointing a coding assistant at the many **examples** we have created.
- 3 • And **look** out for what you don't have to decide.
- 4 • That's normally the **expensive** part of making something reusable.
- 5 I'd like to thank my four **collaborators** for their contributions – and you for your attention.
- 6 Unfortunately we don't get a **Q&A**.
- 7 • So if you have **questions** for me, want to see or hear more, come find me.